

Algorithm: Square of a number

Input a number : real

Output The square of the number : real

```
1 begin
```

```
2 | print("Enter a real number :")*    #Préparation du traitement
```

```
3 read(value) #Instruction pour lire ...
```

```
4 square ← value × value           #Traitement, opération ...
```

```
5 print("the square of ", value)
```

```
6 print("is ", square)           #Affichage du résultat ...
```

7 end

Commenter : anglais, déclarations, entrées, sorties, traitements

Les trois étapes d'un algorithme

1 Préparation du traitement

- données nécessaires à la résolution du problèmes
- paramètres

② Traitement

- résolution pas à pas, après décomposition en sous-problèmes si nécessaire
- opérations mathématiques ou sur les chaînes de caractères, ou sur les fichiers, ou des données

3 Edition des résultats

- sortie standard : l'écran
- sortie dans un fichier

Schéma à mémoriser et à utiliser à chaque fois qu'on doit programmer.

Déclarer une variable

variable $\langle$ liste d'identificateur $\rangle$: **type**

- **Fonction**

- réserver l'espace mémoire pour stocker les données
- définir le type : entiers, réels, caractères, booléen

- **Examples**

```
variables  value, square      : real
            i, index         : integer
            flag             : boolean
            name, city       : strings (chaîne de caractères)
```

Séparateur de variables : la virgule (,), déclaration : deux points (:)

- On préférera plutôt déclarer le type de la variable : **Input** ou **Output**
- La mise en forme et la convention peuvent légèrement varier suivant les auteurs,

Afficher ou imprimer des données, messages, résultats

```
print ( < liste de variables, chaines de caractères, ... > )
```

- **Fonction**

- imprimer ou afficher une ou plusieurs informations ou messages
- mettre des messages clairs et instructifs

- **Examples**

```
print ("la valeur de l'entrée est ", value)
```

```
print ("j'habite à ", city, " qui se trouve en ", country)
```

- Notez les "" à cause de l'apostrophe utilisée en français
- Affichage de chaînes de caractères et de contenu de variables.

Saisir une donnée

```
read (⟨ liste de noms de variables ⟩)
```

- **Fonction**

- placer en mémoire les données entrées
- il faut toujours mettre un message avant pour préciser les entrées à l'utilisateur

- **Examples**

```
print ("Entrer un nombre réel positif : ")
```

read (value)

```
read (i, index)
```

read (city, country)

Définir des constantes et leur valeur

parameter ($\langle$ identificateur $\rangle$: type) $\leftarrow$ $\langle$ valeur ou expression $\rangle$

- **Fonction**

- un paramètre ou une constante a sa valeur fixée (figée)
- son traitement peut être légèrement différent d'une variable

- **Examples**

```
parameter (PI : real) ← 3.1415927
```

parameter (IMAX : integer) $\leftarrow 2 \times 11 + 5$

- une constante est toujours un paramètre d'entrée
- par conventionn, le nom de la constante est **en lettres capitales**

Affecter une valeur à une variable

$$\langle \text{identificateur} \rangle \leftarrow \langle \text{expression ou valeur ou identificateur} \rangle$$

- **Fonction**

- attribution d'une valeur à une variable dans la mémoire de l'ordinateur
- attention au sens de la flèche.

- **Examples**

```
value ← 3.1415927
```

```
result ← 2 × value + 5
```

$$\text{value} \leftarrow 2 \times \text{value} + 5$$

```
name ← "no_one"
```

- $\leftarrow$ est l'opérateur d'affectation en algorithmique
- il peut changer d'un langage de programmation à un autre
- c'est extrêmement souvent "="

Effectuer une opération sous condition "si"

Algorithm: Condition "si"

Une condition est un test dont le résultat est un booléen "vrai" ou "faux" (true/false)

```
if cond1 then
  | print("do something")
else
  | print("do something else")
end
```

```

if cond1 then
  | cond1 is true
else if cond2 then
  | cond1 is false but cond2 is true
else
  | all is wrong
end

```

Algorithm: Condition "si" emboîtée

On peut ajouter des niveaux dans les tests

```

1  begin
2      if cond1 then                                #first level
3          | action 1
4      else if cond2 then
5          | if cond 3 and cond 4 then              #second level
6              | action 2
7              else
8              | action 3
9              end
10         else
11         | action 4
12     end
13 end

```

Ne pas faire plus de 2 niveaux, sinon utiliser des fonctions.

Algorithm: Condition "selon"

```
switch value do
  | case (1, 2, 3) do                                #list of numbers
2   |   print(" lower equal to 3")
3   case 4 do
4   |   print("is is four")
5   otherwise do                                     #default case
6   |   print("is is another number !")
end
```

- On peut le faire avec des mots (chaîne de caractères)
- Très utile pour faire des menus
- Cette instruction n'existe pas dans tous les langages (bientôt dans Python)

Algorithm: Boucle "Pour"

```

1 for counter  $\leftarrow$  init_value to final_value by step do           #counter is an
   integer
2   | do something
3 end
4 for value  $\in$  list do                                             #value in a given list
5   | do something else
6 end

```

- Répétition d'une tâche un certain nombre de fois.
- Ne fonctionne que **si on connaît à l'avance** combien d'itérations seront effectuées
- **Ne jamais modifier la valeur du compteur** → ERREUR
- La variable de la boucle peut être comprise dans une liste de valeurs ou d'éléments (très puissant dans Python)

Boucle "Tant que"

Algorithm: Boucle "tant que"

```
i ← 2                                #Initialisation required
while i ≤ 1000 do                   #any condition .....
2 |   i ← i × 5
3 end
```

- Répétition d'une tâche un certain nombre de fois tant qu'une certaine condition est vérifiée
- Très utilisé **si on ne connaît pas à l'avance** combien d'itérations seront effectuées
- On doit modifier la condition dans la boucle si on veut en sortir
- La condition est testée **au début** de la boucle, **penser à l'initialiser**
- La boucle peut ne jamais être effectuée
- **À utiliser avec soin et en connaissance**

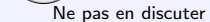
Boucle "Répéter"

Algorithm: Boucle "Répéter"

```
i ← 2                                #Initialisation required
repeat
  | i ← i × 5
until i < 1000
```

- Répétition d'une tâche un certain nombre de fois tant qu'une certaine condition est vérifiée
- Très utilisé si on ne connaît pas à l'avance combien d'itérations seront effectuées
- On doit modifier la condition dans la boucle si on veut en sortir
- La condition est testée **à la fin** de la boucle, **penser à l'initialiser**
- La boucle est **effectuée au moins une fois**
- **À utiliser avec soin et en connaissance**

1. *Journal of the American Medical Association*, 1997; 277: 1039-1043.



- pour la compréhension
- pour la simplification

Complication

L'algorithme précédent n'est pas réaliste : les enfants voudront distribuer tous les bonbons, et chacun veut distribuer les bonbons. On propose donc une modification.

- À chaque tour l'enfant qui distribue tourne.
Celui qui distribue se sert toujours en dernier.
On distribue N_i bonbons au premier tour, et à chaque tour ce nombre diminue de 1.
Lorsque $N_i = 1$ on ne distribue plus qu'un seul bonbon par tour.
Avec ce processus, il est maintenant possible de distribuer tous les bonbons, mais tous les enfants n'auront pas le m^eme nombre de bonbons.
- Déterminer le nombre de tours, le nombre de bonbons par enfants et qui a distribué en dernier.
- Les enfants seront nommés par un entier de 1 à N_e , une fois pour toute.
- Cette version est plus complexe et nécessite une bonne dose de réflexion et probablement l'emploi de boucles et de listes ou tableaux.

Tableau : structure pour des données de même type

- | | | | | | | | |
|--|--|--|--|--|--|--|--|
| | | | | | | | |
|--|--|--|--|--|--|--|--|

- | | | | | | | | |
|--|--|--|--|--|--|--|--|
| | | | | | | | |
| | | | | | | | |
| | | | | | | | |

- Une **liste** sera un tableau à une dimension, mais dans lequel les données ne seront pas nécessairement de même type.

- un indice (nombre entier positif) permet d'accéder à un élément du tableau
- le premier élément est l'indice 1 (mais dans certains langages cela commence à 0)
- la place prise par le tableau est fonction du nombre d'octets pris par chaque élément en mémoire
- la longueur (taille) du tableau est le nombre d'éléments, à ne pas confondre avec sa forme ($n \times m$ en 2D)

-5	4	2.3	1	-1.4	101
1	0	1	1	0	0
r	e	q	u	i	n

Matlab

Python

Notations (en fonction des conventions) :

- 1 D : $x(1)$ ou $x[1]$
- 2 D : $z(2, 3)$ ou $z[2, 3]$

Dans les langages de programmation $x(1)$ est différent de $x[1]$.

Résumé, exemple

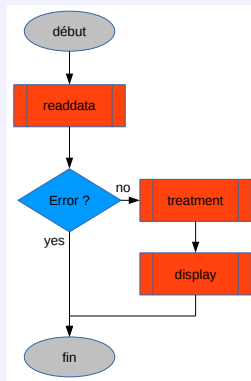
- simplifier l'écriture pour être plus compréhensible
- déporter des parties de l'algorithme ailleurs
- l'algorithme est plus lisible, donc plus clair et facile à modifier ou implémenter

Algorithm: enter array values

Output n : integer

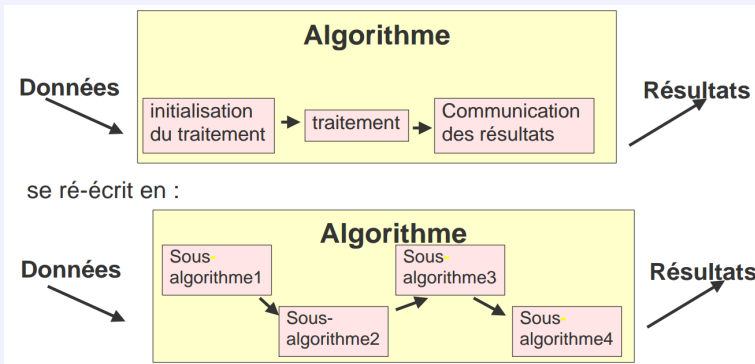
Output tab : array [1, 10] integer

```
0 begin
1   readdata(n, error)
2   if not error then
3       treatment(n, tab)
4       display(n, tab)
5   end
6 end
```



Les nouvelles fonctions sont alors appelées des **sous-algorithmes**

Réorganisation algorithmique



- le sous-algorithme effectue le traitement sur des éléments bien **définis** et **délimités**, et si possible **indépendamment** du contexte de l'algorithme appelant
- un sous-algorithme peut en appeler un autre.

100

Function *treatment*(*n*)

Input n : integer

```
#Maximal length of the array
```

Output *tab* : array [1, 10] integer

0	variable i : integer
---	-------------------------------

```
#Local variable for the loop
```

1	begin
---	-------

```
2      for  $i = 1$  to  $n$  do
```

```
3 print("enter element (integer) ", i)
```

4			<code>read(<i>tab</i>[<i>i</i>])</code>
---	--	--	---

5	$tab[i] \leftarrow 3 * tab[i] + 2$
---	------------------------------------

6		end
---	--	-----

7	return tab
---	------------

8 | end

1. *What is the purpose of this study?*

D $\frac{1}{2} \times \frac{1}{2} = \frac{1}{4}$

```

1 // read the size of the input array

```

```
#Maximal length of the array
```

```
#length of the filled part of the array
```

[illegible][illegible]

0		1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465
---	--	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99	100	101	102	103	104	105	106	107	108	109	110	111	112	113	114	115	116	117	118	119	120	121	122	123	124	125	126	127	128	129	130	131	132	133	134	135	136	137	138	139	140	141	142	143	144	145	146	147	148	149	150	151	152	153	154	155	156	157	158	159	160	161	162	163	164	165	166	167	168	169	170	171	172	173	174	175	176	177	178	179	180	181	182	183	184	185	186	187	188	189	190	191	192	193	194	195	196	197	198	199	200	201	202	203	204	205	206	207	208	209	210	211	212	213	214	215	216	217	218	219	220	221	222	223	224	225	226	227	228	229	230	231	232	233	234	235	236	237	238	239	240	241	242	243	244	245	246	247	248	249	250	251	252	253	254	255	256	257	258	259	260	261	262	263	264	265	266	267	268	269	270	271	272	273	274	275	276	277	278	279	280	281	282	283	284	285	286	287	288	289	290	291	292	293	294	295	296	297	298	299	300	301	302	303	304	305	306	307	308	309	310	311	312	313	314	315	316	317	318	319	320	321	322	323	324	325	326	327	328	329	330	331	332	333	334	335	336	337	338	339	340	341	342	343	344	345	346	347	348	349	350	351	352	353	354	355	356	357	358	359	360	361	362	363	364	365	366	367	368	369	370	371	372	373	374	375	376	377	378	379	380	381	382	383	384	385	386	387	388	389	390	391	392	393	394	395	396	397	398	399	400	401	402	403	404	405	406	407	408	409	410	411	412	413	414	415	416	417	418	419	420	421	422	423	424	425	426	427	428	429	430	431	432	433	434	435	436	437	438	439	440	441	442	443	444	445	446	447	448	449	450	451	452	453	454	455	456	457	458	459	460	461	462	463	464	465	466
---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----	-----

[illegible][illegible]

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47	48	49	50	51	52	53	54	55	56	57	58	59	60	61	62	63	64	65	66	67	68	69	70	71	72	73	74	75	76	77	78	79	80	81	82	83	84	85	86	87	88	89	90	91	92	93	94	95	96	97	98	99
---	---	---	---	---	---	---	---	---	---	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----	----

- PDF GENERATED BY www.pdfcrowd.com